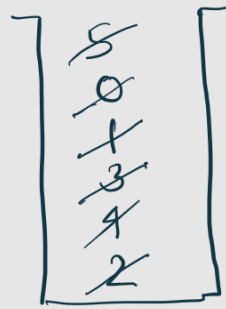
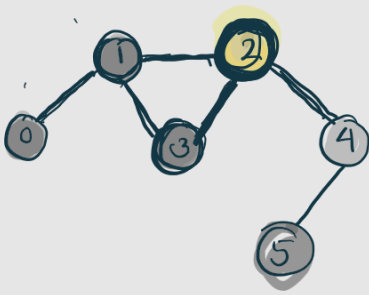


Do DFS → stack

Source Node = 2

DFS:  
white

0: white  
1: Grey  
2: Black



output:  
2 1 0 3 4 5

color:

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 2 | 2 | 2 | 2 |   | 2 |
| 0 | 1 | 2 | 3 | 4 | 5 |

2 1 0 3 4 5

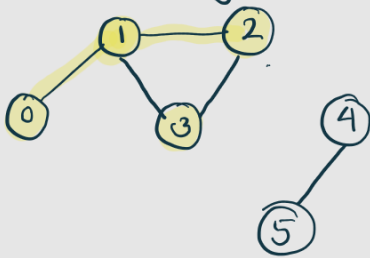
$O(V+E)$

Pseudocode:

```
static int[] color;
void dfs (int curr) {
    color[curr] = 1;
    for (i = 0; i < adj[curr].size(); i++) {
        int next = adj[curr].get(i);
        if (color[next] == 0) {
            dfs(next);
        }
    }
    color[curr] = 2;
}
```

$O(V^2)$   
↑  
worst case

DFS for disconnected graph



//main

```
for (i = 0; i < n; i++) {
    if (color[i] == 0) {
        dfs(i);
        count++;
    }
}
```

```
static int[] color;
void dfs (int curr) {
    color[curr] = 1;
    for (i = 0; i < adj[curr].size(); i++) {
        int next = adj[curr].get(i);
        if (color[next] == 0) {
            dfs(next);
        }
    }
    color[curr] = 2;
}
```



|   |   |   |   |   |
|---|---|---|---|---|
|   |   | # |   | # |
| # |   |   |   | # |
|   |   |   |   |   |
| # | # |   |   | # |
|   |   |   | # | # |

# = Land

Find the number of land

DFS:

① color = 0

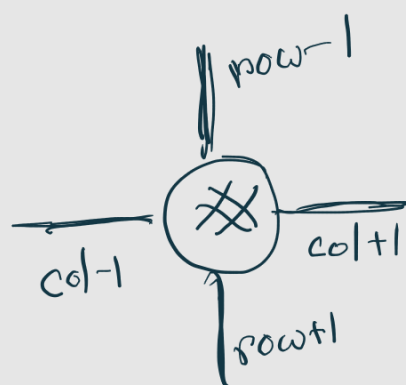
② #

// main

```
for (curr_r = 1; curr_r <= row; curr_r++) {
```

```
    for (curr_col = 1; curr_col <= col; curr_col++) {
```

```
        if (color[curr_r][curr_col] == 0 && arr[curr_r][curr_col] == '#') {
            dfs(arr, curr_r, curr_col);
            count++;
        }
    }
}
```



```
static int dr = {1, -1, 0, 0}
```

```
static int dc = {0, 0, 1, -1}
```

```
void dfs(char[][] arr, int r, int c) {
```

```
    color[r][c] = 1;
```

```
    for (i = 0; i < 4; i++) {
```

```
        int nr = r + dr[i];
```

```
        int nc = c + dc[i];
```

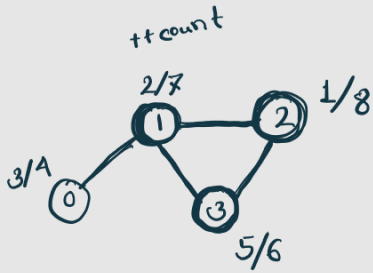
```
        if (nr >= 1 && nr <= R && nc >= 1 && nc <= C) {
```

```
            if (color[nr][nc] == 0 && arr[nr][nc] == '#') {
                dfs(arr, nr, nc);
            }
        }
    }
```

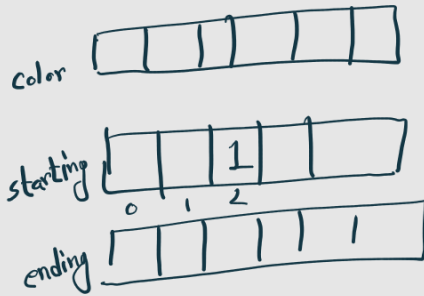
```
    color[r][c] = 2
```

```
}
```

Find starting time (discovery time) and end time



start node 2

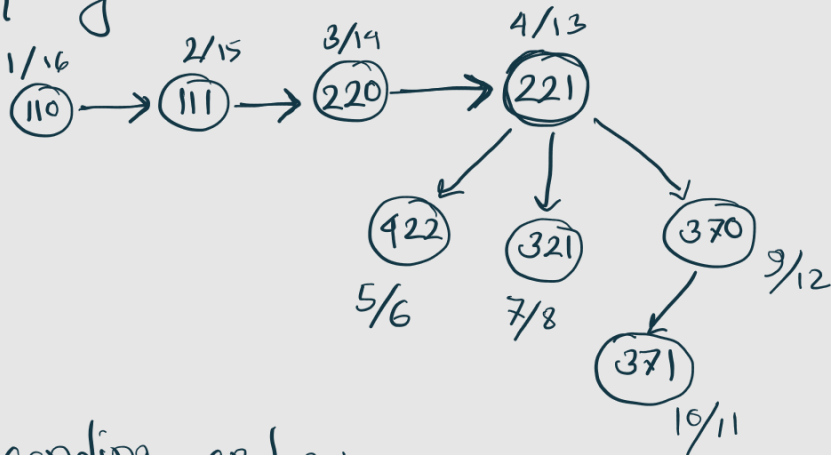


```
static int[] color;
static int[] starting;
static int[] ending;
static int count;
void dfs(int curr){
    color[curr]=1;
    starting[curr]=++count;
```

```
    for(i=0; i<adj[curr].size(); i++){
        int next = adj[curr].get(i);
        if (color[next]==0){
            parent[next]=curr;
            dfs(next);
        }
    }
```

```
    color[curr]=2;
    ending[curr]=++count;
}
```

Topological sort



\* Sequence isn't always the same

Descending order: 16, 15, 14, 13, 12, . . . . .

Sorting: 110, 111, 220, 221, 370 . . . . .

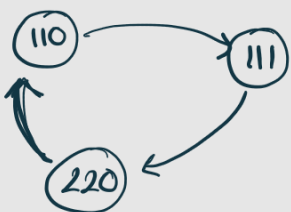
☐



(X) Undirected

Directed (✓)

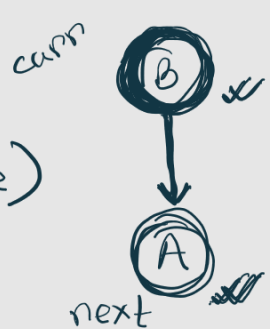
\* Only works for DAG (Directed Acyclic Graph)



cyclic (X)

## Edge classification

① Tree Edge:  $(\text{curr\_node}) \xrightarrow{\text{Grey}} (\text{next\_node}) \xrightarrow{\text{white}}$



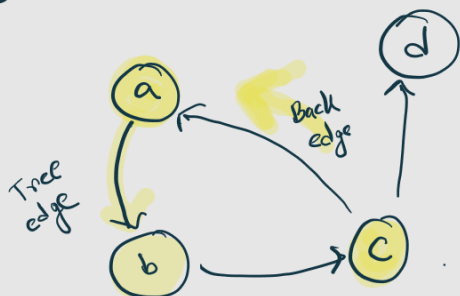
② Back Edge:  $(\text{curr\_node}) \xrightarrow{\text{Grey}} (\text{next\_node}) \xrightarrow{\text{Grey}}$   
\*Cycle

③ Forward Edge:  $(\text{curr\_node}) \xrightarrow{\text{Grey}} (\text{next\_node}) \xrightarrow{\text{Black}}$   
 $d[u] < d[v]$



④ Cross Edge:  $(\text{curr\_node}) \xrightarrow{\text{Grey}} (\text{next\_node}) \xrightarrow{\text{Black}}$   
 $d[u] > d[v]$

## Cycle detection



```

static int[] color;
static boolean found = false;
  
```

```

void dfs(int curr){
  
```

```

    color[curr] = 1;
  
```

```

    for(i=0; i < adj[curr].size(); i++){
  
```

```

        int next = adj[curr].get(i);
  
```

```

        if (color[next] == 0){
  
```

```

            dfs(next);
  
```

```

        }
  
```

```

        else if (color[next] == 1){
  
```

```

            found = true;
  
```

```

            return;
  
```

```

        }
  
```

```

    }
  
```

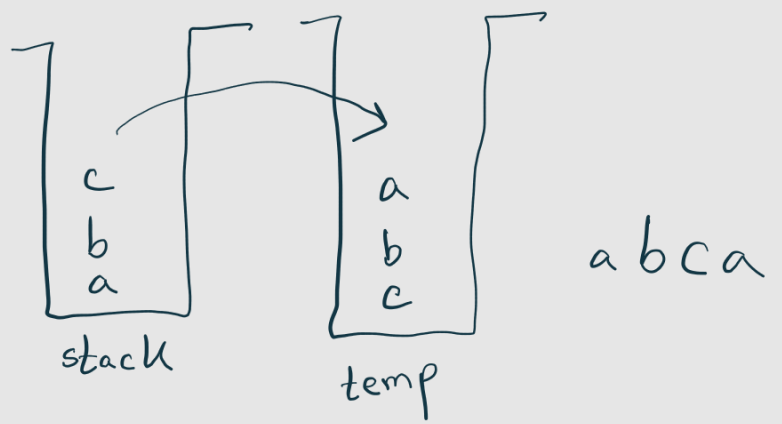
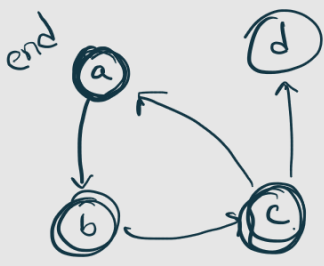
```

    color[curr] = 2;
  
```

```

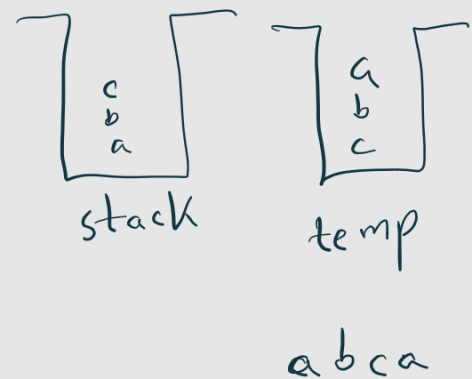
}
  
```

## Print cycle



```
static int[] color,
static boolean found = false;
static Stack<Integer> stack = new Stack<>();
static int end;
```

```
void dfs(int curr){
    color[curr] = 1;
    stack.push(curr);
    for(i = 0; i < adj[curr].size(); i++){
        int next = adj[curr].get(i);
        if (color[next] == 0){
            dfs(next);
        }
        else if (color[next] == 1){ // Back edge
            found = true;
            end = next;
            cycle(end); ✓
            return;
        }
    }
    stack.pop();
    color[curr] = 2;
}
```



```
void cycle(int end){
    Stack<Integer> temp = new Stack<>();
    while(!stack.isEmpty()){
        int x = stack.pop();
        temp.push(x);
        if (x == end) break;
    }
```

```
while(!temp.isEmpty()) s.o.p(temp.pop() + " ");
s.o.p(end);
}
```